Question 1: Mixture of Bernoulli products

Task 1:
Define latent variable as $z_{nk} \in \{0, 1\}$ such that $z_{nk} = 1$ if x_n belongs to cluster k, and $\sum_{k=1}^{K} z_{nk} = 1$. The model parameters θ are π_k and $\mu_{k,d} \in [0, 1]$ The likelihood of observing both $\mathbf{x}$ and $\mathbf{z}$ is

$$p(\mathbf{x}, \mathbf{z} \mid \theta) = \prod_{n=1}^{N} \prod_{k=1}^{K} \left[\pi_k \prod_{d=1}^{D} \mu_{k,d}^{x_{nd}} (1 - \mu_{k,d})^{1 - x_{nd}} \right]^{z_{nk}}$$

Taking the logarithm, we obtain the complete-data log-likelihood:

$$\mathcal{L}(\theta) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \left(\log \pi_k + \sum_{d=1}^{D} [x_{nd} \log \mu_{k,d} + (1 - x_{nd}) \log(1 - \mu_{k,d})] \right)$$

For the E-step, fix θ and compute responsibilities $\gamma(z_{nk})$

$$\gamma(z_{nk}) = \frac{\pi_k p(x_n \mid \mu_k)}{\sum_{j=1}^{K} \pi_j p(x_n \mid \mu_j)}, \quad \text{where} \quad p(x_n \mid \mu_k) = \prod_{d=1}^{D} \mu_{k,d}^{x_{nd}} (1 - \mu_{k,d})^{1 - x_{nd}}$$

For the M-step, fix $\gamma(z_{nk})$ and update parameters by maximising $\mathcal{L}(\theta)$. The first order optimality conditions give:

$$N_k = \sum_{n=1}^{N} \gamma(z_{nk}), \quad \pi_k^{new} = \frac{N_k}{N}, \quad \mu_{k,d}^{new} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) x_{nd}$$

Task 2:
To ensure numerical stability and prevent underflow. The likelihood $p(x_n \mid \mu_k)$ involves the product of D probabilities. Since these values are less 1, multiplying them might result in a number smaller than machine precision, which rounds to zero and causes division by zero in $\gamma(z_{nk})$. Working in log-space converts this unstable product into a stable sum:

$$\log p(x_n \mid \mu_k) = \sum_{d=1}^{D} [x_{nd} \log \mu_{k,d} + (1 - x_{nd}) \log(1 - \mu_{k,d})]$$

This allows the use of the LogSumExp to safely normalise the probabilities without overflow or underflow.
Task 3:

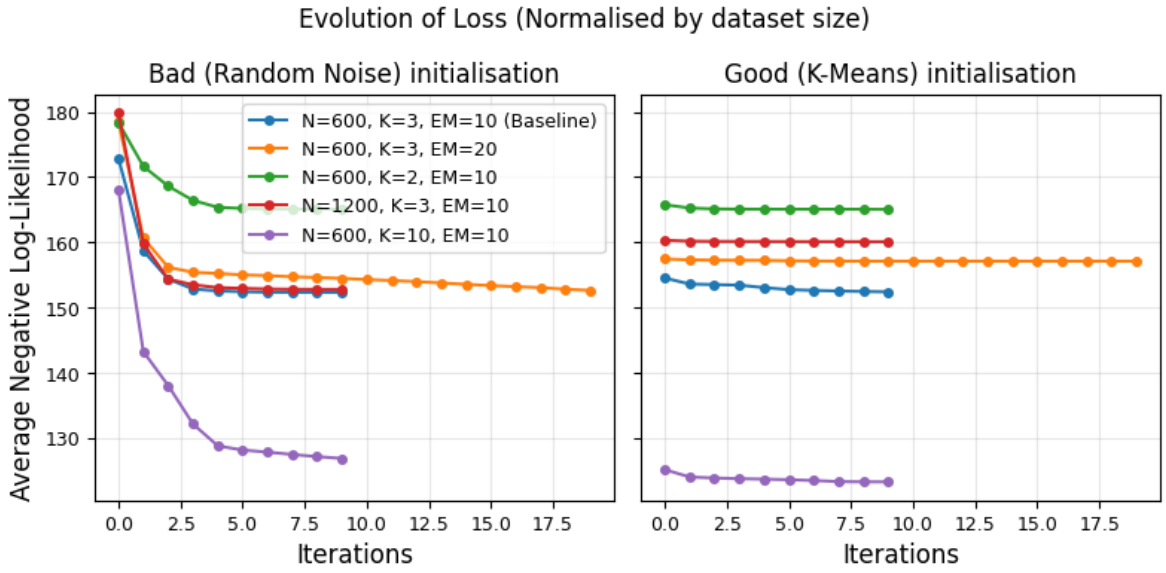


Figure 1: Evolution of the loss

Increasing training samples or the number of iterations result in minimal improvements. The best fit is obtained by choosing $K = 10$. Although the training set contains only 3 unique digits, choosing higher K

allow the model to capture different writting styles like slanted versus straight, rather than just the average shape of the class itself.

Task 4:

The experiments confirm that EM is strictly monotonic, with the loss decreasing at every iteration. However, the trajectory is highly sensitive to initialisation. Using a good initialisation like K-Means allows the EM to start at a much lower loss and converge almost immediately. In contrast, random initialisation forces the model to spend the first 5 iterations finding the general location of the clusters.

Task 5:

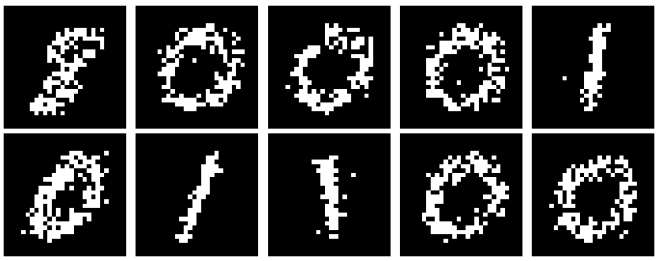
From task 1, the M-step update rule $\mu_{k,d} = \frac{1}{N_k}\sum_{n=1}^{N}\gamma(z_{nk})x_{nd}$ defines μ_k as the weighted average of all images, where the weights are proportional to the responsibility. μ_k represents the centre of each cluster k. Images with high responsibility ($\gamma \approx 1$) contribute strongly to this mean, while those with low responsibility ($\gamma \approx 0$) are ignored.

To generate an image, the model iterates through every pixel d and samples a binary value from a Bernoulli distribution parameterized by $\mu_{k,d}$. The value of μ also encodes the model's uncertainty, as the variance of a Bernoulli variable is $\mu(1-\mu)$. Pixels where $\mu \approx 0$ (black) or $\mu \approx 1$ (white) implies that variance is near zero, representing the main features where the model is certain. Pixels where $\mu \approx 0.5$ (gray) represent regions of maximum variance. These are writing styles feature like stroke thickness or slant where the model is uncertain.

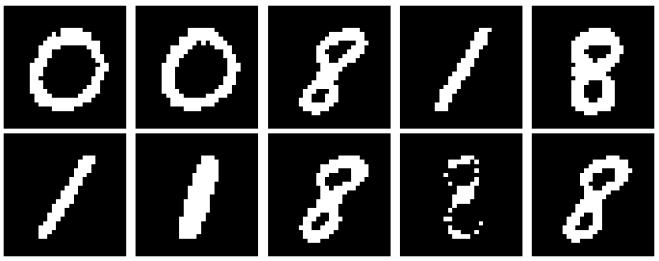
Task 6:

To sample from the learnt model, ancestral sampling is implemented. First, sample $z \sim p(z)$ by picking a cluster k based on weights π. Then sample $x \sim p(x|z)$ by drawing each pixel x_d independently from a Bernoulli distribution with parameter $\mu_{k,d}$

From figure 2a, the samples can be recognised as digts $(0, 1, 8)$, meaning that the model has learned the global structure of the classes. However, the images are very noisy compared to the traininig data. The poor quality is due to the independence assumption in the likelihood function $p(x \mid z) = \prod_{d=1}^{D} p(x_d \mid z)$. Real image has high correlation because the stroke are smooth and continous. BMM model does not capture this correlation. If two neighboring pixels A and B both have a high probability of being white ($\mu = 0.9$), there is still a chance that pixel A get value 1 and pixel B get 0, creating the disconnected pattern observed in the samples.



(a) Task 6 result



(b) Task 7 result

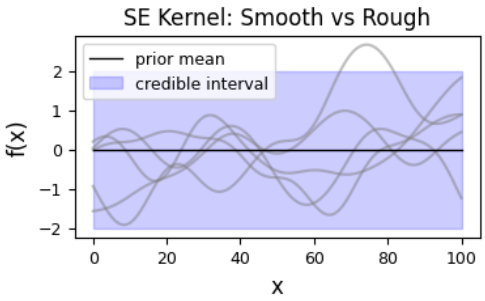
Figure 2: Images generated by the learned model

Task 7:

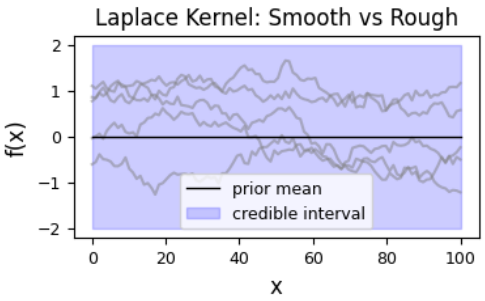
From figure 2b, the digits now look less noisy and sharper. By using temperature parameter $\alpha = 10$, it push the pixel probabilities toward certainty (0 or 1). However, the trade-off is that the stochasticity is reduced, the model now produce almost identical output, and less creative. High temperature can also make bad clusters look worse. Looking at the 4th image at the bottom row, it still have some discontinuity. This is because for gray pixels ($\mu = 0.5$), when α is applied, these uncertain gray areas are forced to be either 0 or 1.

Question 2: Gaussian processes

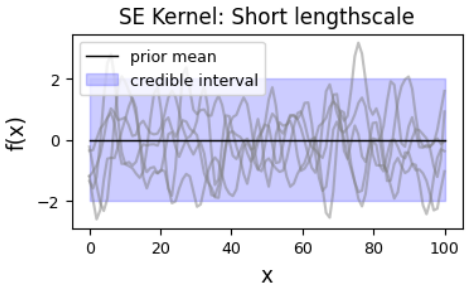
Task 2:



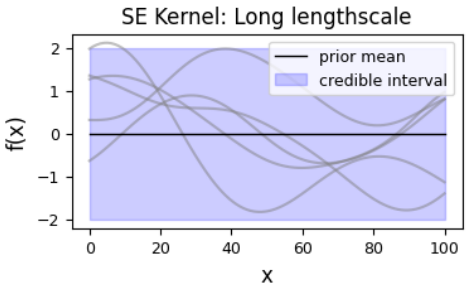
(a) SE Kernel



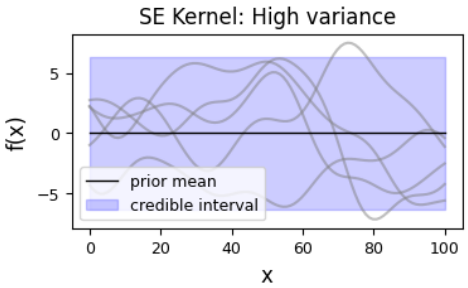
(b) Laplace Kernel



(c) SE with short lengthscale



(d) SE with long lengthscale



(e) SE with high lengthscale

Figure 3: Trajectories from the GP prior

The plots generated using the SE kernel show smooth curves and infinitely differentiable trajectories. This is because from the kernel definition, similarity is assessed based on the squared Euclidean distance between points, so correlation between two points decays smoothly. In contrast, the plots generated using the Laplace kernel show rough, jagged trajectories. These functions are continuous but non-differentiable. This behavior arises because the Laplace kernel models similarity based on the absolute distance $|x - x'|$. The singularity in the absolute value function at the origin allows the process to change direction instantly.

Using SE kernel, short lengthscale show the functions become highly oscillatory because the correlation between two points decays very rapidly. With a short memory, the value at $x = 0$ shares very little similarity with the value at $x = 0.5$, leading to rapid fluctuations. In contrast, long lengthscale reduce the frequency where the function value at a given point remains highly correlated with values significantly far away. Additionally, varying the marginal variance σ^2 scales the vertical amplitude of the functions and widens the credible interval.

In all generated plots, the shaded 95% credible interval appears as a horizontal band of constant width. This confirms that both the SE and Laplace kernels are stationary. Because these kernels depend only on the difference of their arguments $x - x'$, the covariance is invariant under translations in the input space. Consequently, the marginal variance σ^2 is constant everywhere in the prior.

Task 4:

Dataset Size	SE Kernel NLL	Laplace Kernel NLL
10% (Small)	458.766	461.664
50% (Medium)	107.119	107.055
90% (Large)	60.181	60.978

Table 1: Negative Log-Likelihood (NLL) comparison for SE and Laplace kernels across dataset sizes

The performance of both kernels is very similiar across all experiments. When training data increases from 10% to 90%, the NLL decreases significantly. This indicates that the model fits the underlying process with higher confidence and accuracy as more observations become available.

When the lengthscale is small ($l = 0.1$), both kernels failed to converge the global optimum. SE kernel became trapped in a poor local minimum with an NLL of ≈563.5, which is worse than baseline ≈459. When

the lengthscale is large ($l = 100$), the SE kernel managed to converge back to the baseline NLL. However, the Laplace kernel failed to fully recover, getting stuck at poor local minimum with slightly higher NLL 476. Both kernels showed high robustness to the initialisation of marginal variance $\sigma^2 = \{0.1, 100\}$ and noise variance $\sigma_\epsilon^2 = \{0.1, 100\}$. Varying these parameters did not prevent the optimizer from reaching the optimal baseline NLL. In conclusion, SE kernel is the preferred option for this dataset.

Task 6:

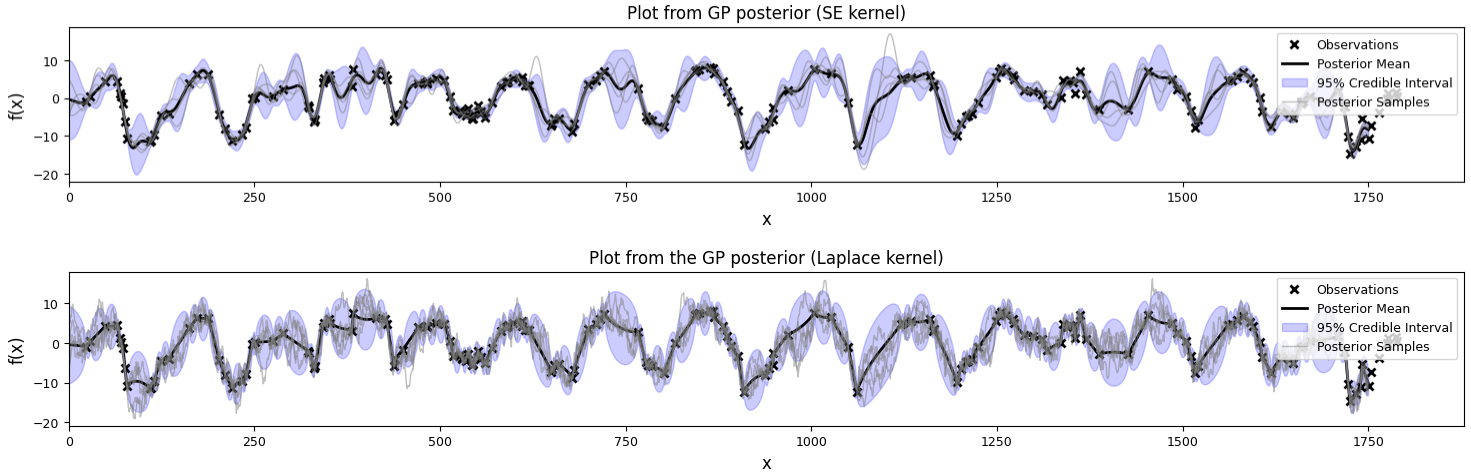


Figure 4: Trajectories from the GP posterior

The top plot (SE kernel) shows that posterior mean (black line) and the random samples (grey lines) are smooth, whereas the bottom plot (Laplace kernel) are jagged and spiky. SE kernel tries to fit a smooth interpolation through the points and treating sharp spikes in the data as measurement noise. The roughness in Laplace kernel allows the model to change direction instantly, treating sharp spikes in the data as true signal. This makes the Laplace kernel qualitatively better data with rapid fluctuations.

In regions where data is sparse, around $x \approx 1000 - 1100$, both kernels show wider uncertainty. The SE kernel produces credible intervals that expand and contract smoothly. The Laplace kernel produces intervals that are pinched at observation points, and expand rapidly between them. This reflects the assumption that the function value can drift away from the mean more abruptly than the smooth constraints of the SE kernel.

Task 7:

Metric	SE Kernel	Laplace Kernel
NLL	458.8	461.7
MSE	3.84	3.37

Table 2: Comparison of SE and Laplace kernels predictive performance based on NLL and MSE

In terms of NLL, the SE Kernel achieved a lower value of 458.8 compared to 461.7 for the Laplace kernel. The minimisation of NLL consists of two terms:

$$\text{NLL} = \frac{1}{2}\mathbf{y}^\top(\mathbf{C})^{-1}\mathbf{y} + \frac{1}{2}\log|\mathbf{C}|, \quad \text{where} \quad \mathbf{C} = \mathbf{K} + \sigma_\epsilon^2\mathbf{I}$$

On the sparse training data (10%), SE kernel treats the signal's roughness as measurement noise, attributing the jumps between points to an inflated noise variance σ_ϵ^2. Although this increases the $\log|\mathbf{C}|$ term, it also reduces the quadratic datafit term $\mathbf{y}^\top(\mathbf{C})^{-1}\mathbf{y}$, resulting in a lower overall NLL. However, the difference between the two NLL values is relatively small ($\approx 0.6\%$).

In terms of MSE, the Laplace kernel achieved a lower MSE of 3.37 compared to 3.84 for the SE kernel. Since the MSE was calculated on the full dataset where 90% of points were unseen during training, this metric tests the interpolation capability of the models and punishes models that smooth over the true sharp peaks of the signal.

Evaluating both metrics is not redundant because they measure different properties on different scales. NLL is a probabilistic metric evaluated only on the 10% training subset, while MSE is a deterministic metric evaluated across the entire 100% dataset. Overall, the Laplace kernel, which can model rough processes, was able to track the rapid fluctuations in the missing data better than the SE kernel.

Question 3: Recovering EM from VI

We start by writing ELBO for the joint distribution of latent variables z and parameters θ, and substitute the factorised variational family $q(z, \theta) = q_z(z)\delta(\theta - \theta_0)$

$$\begin{aligned}\text{ELBO} &= \iint q(z,\theta) \log p(x,z,\theta)\, dzd\theta - \iint q(z,\theta) \log q(z,\theta) dzd\theta \\ &= \iint q_z(z)\delta(\theta-\theta_0) \log p(x,z,\theta)\, dzd\theta - \iint q_z(z)\delta(\theta-\theta_0) \log[q_z(z)\delta(\theta-\theta_0)] dzd\theta\end{aligned}$$

For E-step, fix θ_0 and update $q_z(z)$. Using CAVI update rule, the optimal distribution for q_z is proportional to the expectation of the joint probability with respect to other variables q_θ:

$$\log q_z^*(z) \propto_{q_\theta} \mathbb{E}_{q_\theta}[\log p(x,z,\theta)]$$

Using the sifting property of Dirac delta function $\int f(\theta)\delta(\theta-\theta_0)d\theta = f(\theta_0)$, the expectation simply evaluates the function at θ_0:

$$\mathbb{E}_{q_\theta}[\log p(x,z,\theta)] = \int \delta(\theta-\theta_0) \log p(x,z,\theta)\, d\theta = \log p(x,z,\theta_0)$$

Therefore:

$$\log q_z^*(z) \propto \log p(x,z,\theta_0) \propto \log p(x,z \mid \theta_0) q_z^*(z) \propto p(x,z \mid \theta_0)$$

To normalize this distribution, we divide by the marginal likelihood

$$q_z^*(z) = \frac{p(x,z \mid \theta_0)}{\int p(x,z \mid \theta_0) dz} = p(z \mid x, \theta_0)$$

Thus, minimising the KL divergence recovers the exact posterior $p(z \mid x, \theta_0)$, which corresponds to the E-step of the EM algorithm.

For M-step, fix $q_z(z)$ and maximise ELBO with respect to θ_0. Second term of ELBO does not depends on θ_0:

$$\begin{aligned}\text{ELBO Term 2} &= -\iint q_z(z)\delta(\theta-\theta_0) \log[q_z(z)\delta(\theta-\theta_0)]\, dz\, d\theta \\ &= -\iint q_z(z)\delta(\theta-\theta_0) \log q_z(z)\, dz\, d\theta - \iint q_z(z)\delta(\theta-\theta_0) \log \delta(\theta-\theta_0)\, dz\, d\theta \\ &= -\int q_z(z) \log q_z(z)\, dz - \int \delta(\theta-\theta_0) \log \delta(\theta-\theta_0)\, d\theta \quad (\text{Use} \int q_z(z)dz = 1) \\ &= -\int q_z(z) \log q_z(z)\, dz - \int \delta(u) \log \delta(u)\, du \quad (\text{Use } u = \theta - \theta_0, du = d\theta)\end{aligned}$$

We only use the first term of ELBO that depends on θ_0:

$$L(\theta_0) = \iint q_z(z)\delta(\theta-\theta_0) \log p(x,z,\theta)\, \mathrm{d}z\, \mathrm{d}\theta$$

Integrate out θ using the Dirac delta sifting property:

$$L(\theta_0) = \int q_z(z) \log p(x,z,\theta_0)\, \mathrm{d}z = \mathbb{E}_{q_z(z)}[\log p(x,z,\theta_0)]$$

The optimisation becomes:

$$\theta_0^* = \arg\max_{\theta_0} \left(\mathbb{E}_{q_z}[\log p(x,z \mid \theta_0)] + \log p(\theta_0)\right)$$

This recovers the M-step. If the prior $\log p(\theta_0)$ is assumed constant, this is equivalent to maximum likelihood estimation. If we include the prior term, this recovers maximum a posteriori estimation.

Question 4: KL divergence as a particular case

Let the numerator and denominator be $N(\alpha)$ and $A(\alpha)$ respectively:

$$N(\alpha) = 1 - \int p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}} dx, \quad A(\alpha) = 1 - \alpha^2$$

As $\alpha \to \pm 1$, both $N(\alpha) \to 0$ and $A(\alpha) \to 0$, giving $\frac{0}{0}$, so we use L'Hôpital's rule:

$$\lim_{\alpha \to \pm 1} D_\alpha(p\|q) = 4 \lim_{\alpha \to \pm 1} \frac{N'(\alpha)}{A'(\alpha)}$$

Take derivative with respect to α:

$$A'(\alpha) = \frac{d}{d\alpha}(1 - \alpha^2) = -2\alpha$$

For the numerator, assume finite KL divergences so we can move the derivative inside the integral using Leibniz integral rule, to compute

$$N'(\alpha) = -\int \frac{\partial}{\partial \alpha} \left(p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}} \right) dx$$

Let $f(x, \alpha) = p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}}$. Take the log to simplify:

$$\log f(x, \alpha) = \frac{1-\alpha}{2} \log p(x) + \frac{1+\alpha}{2} \log q(x)$$

$$\frac{\partial}{\partial \alpha} \log f(x, \alpha) = -\frac{1}{2} \log p(x) + \frac{1}{2} \log q(x) = \frac{1}{2} \log \frac{q(x)}{p(x)}$$

$$\frac{\partial f}{\partial \alpha} = f(x, \alpha) \frac{1}{2} \log \frac{q(x)}{p(x)}$$

$$N'(\alpha) = -\frac{1}{2} \int p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}} \log \frac{q(x)}{p(x)} \mathrm{d}x$$

For case $\alpha \to 1$:

$$A'(1) = -2(1) = -2$$

$$N'(1) = -\frac{1}{2} \int p(x)^0 q(x)^1 \log \frac{q(x)}{p(x)} \mathrm{d}x = -\frac{1}{2} \int q(x) \log \frac{q(x)}{p(x)} \mathrm{d}x = -\frac{1}{2} \mathrm{KL}(q\|p)$$

$$\lim_{\alpha \to 1} D_\alpha(p\|q) = 4 \frac{-\frac{1}{2} \mathrm{KL}(q\|p)}{-2} = \mathrm{KL}(q\|p)$$

For case $\alpha \to -1$:

$$A'(-1) = -2(-1) = 2$$

$$N'(-1) = -\frac{1}{2} \int p(x)^1 q(x)^0 \log \frac{q(x)}{p(x)} \mathrm{d}x = \frac{1}{2} \int p(x) \log \frac{p(x)}{q(x)} \mathrm{d}x = \frac{1}{2} \mathrm{KL}(p\|q)$$

$$\lim_{\alpha \to -1} D_\alpha(p\|q) = 4 \frac{\frac{1}{2} \mathrm{KL}(p\|q)}{2} = \mathrm{KL}(p\|q)$$

Hence, α-divergence generalise the KL divergence, recovering the reverse KL when $\alpha \to 1$ and the forward KL when $\alpha \to -1$.